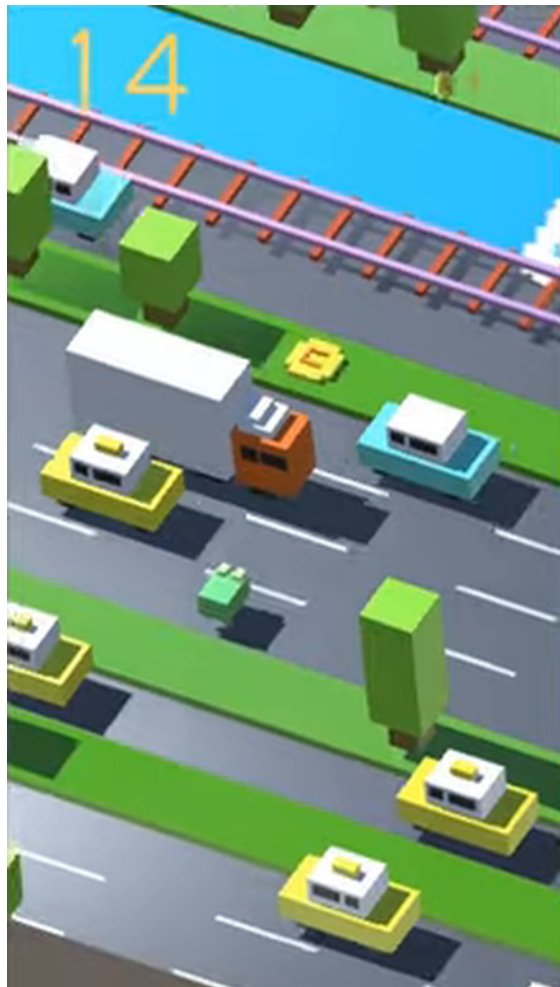


개인 포트폴리오

길건너 친구들

Unity 3D 로 재현한 무한 러너 모작



박 찬 욱

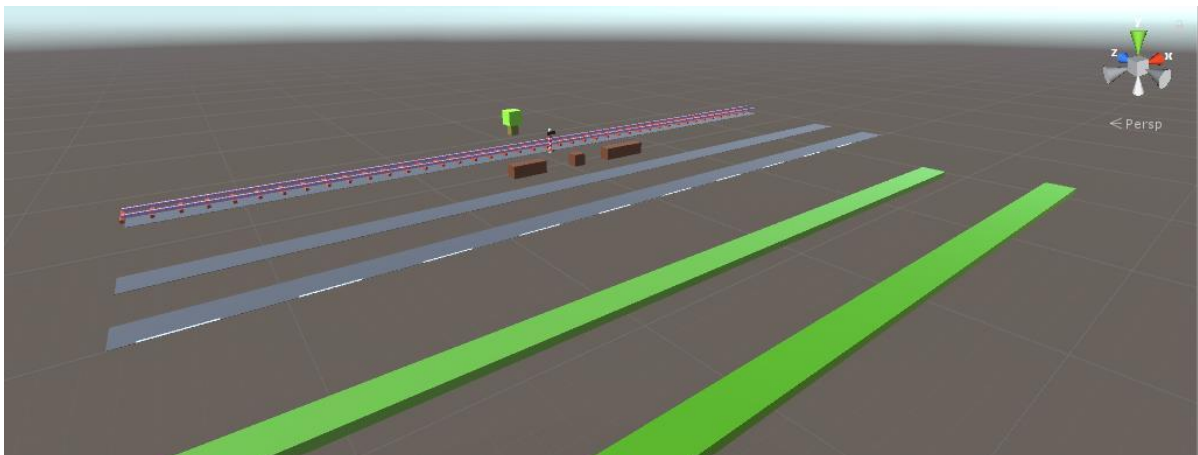
1. 개요

모바일 게임 **Crossy Road (길건너 친구들)**의 핵심 시스템을 코드 레벨에서 분석해 Unity 3D 로 재현한 모작이다. 끝없이 이어지는 지형을 어떻게 만들어내는지, 타일 단위로 움직이는 캐릭터가 어떻게 부드럽게 보이는지, 두 가지가 이 작품의 중심 과제였다.

개발 환경	Unity 2017.1 · C# · 1인 개발
소스 규모	스크립트 16개 / 약 1,230줄
구현 범위	무작위 지형 생성, 타일 단위 이동, 장애물 3종, 점수 · 최고 기록, 사망 연출 3종

구현 기능

- 직전 지형에 따라 다음 지형 후보를 걸러내는 무작위 지형 생성
- 역 선형 보간으로 구한 진행도 하나로 이동 · 점프 · 회전을 함께 구동
- 레이캐스트로 진행 방향의 타일 종류를 먼저 확인하는 이동 판정
- 생성 방식이 서로 다른 장애물 3종 (자동차 · 기차 · 통나무)
- 부딪힌 부위에 따라 달라지는 사망 연출, 물에 빠지는 익사 연출과 카메라 줌



실제 게임에 사용된 지형 프리팹은 이것이 전부다. 이 몇 종류를 규칙에 따라 이어 붙여 무한한 맵을 만든다.

2. 무작위 지형 생성

원작의 맵은 끝없이 이어진다. 하지만 지형을 미리 다 만들어 두는 방식은 로우폴리 게임이라 해도 메모리와 성능 면에서 낭비이고, 무엇보다 **만들어 둔 만큼만** 갈 수 있게 된다. 그래서 플레이어가 전진할 때마다 앞쪽에 지형을 한 줄씩 새로 만드는 방식을 택했다.

그림 1. 플레이어 전진에 맞춰 앞은 생성하고 뒤는 버리는 지형 생성

키메라에 보이는 범위



플레이어가 한 칸 전진할 때마다 Refresh() 가 호출되어 맨 끝에 지형 한 줄을 새로 붙인다. 시작 시 20줄을 미리 깔아 화면을 채우고, 그 뒤로는 필요한 만큼만 만들어 무한한 땅처럼 보이게 한다.

다음에 올 지형은 '올 수 없는 종류를 제외' 하는 방식으로 뽑는다

직전 지형	다음에 올 수 없는 종류 (제외 목록)
차도 시작점	차도 시작점 · 차도 끝점
차도 끝점	물2 · 차도 시작점 · 차도 끝점 · 차선 차도
차선 차도	차도 시작점 · 물1 · 물
물1	물1 · 차선 차도 · 차도 끝점
물	차도 끝점 · 차선 차도

예를 들어 차도 끝점 뒤에 다시 차도 끝점이 오면 차선 없이 도로가 끊겨 통행이 불가능해진다. 이런 조합을 규칙표로 걸러내 지형이 항상 지나갈 수 있는 형태로만 이어지게 했다.

확률 보정 — 물 · 차도가 붙이면 정해진 확률로 같은 종류를 한 줄 더 붙인다. 폭이 1줄로 고정되지 않고 2-3줄로 넓어지는 강과 도로가 나와, 매번 다른 난이도의 지형이 만들어진다.

2.1 통행 불가 조합 걸러내기

단순히 지형 종류를 무작위로 뽑으면 문제가 생긴다. 차선이 없는 **차도 끝점**이 연달아 나오면 도로가 끊겨 지나갈 수 없고, 물이 지나치게 이어지면 통나무 타이밍만으로는 건널 수 없다. `GetRandomStreet()` 은 단순 무작위가 아니라, 직전 지형 종류에 따라 **올 수 없는 종류를 제외 목록으로 만들어 남은 후보 중에서만 뽑는다**.

```
switch (farthestType)
{
    case StreetStatus.Type.CarStreetEnd:
        // 차도 끝점 뒤에 올 수 없는 종류들을 제외 목록에 넣는다
        exceptions.Add((int)StreetStatus.Type.Grass02);
        exceptions.Add((int)StreetStatus.Type.CarStreetStart);
        exceptions.Add((int)StreetStatus.Type.CarStreetEnd);
        exceptions.Add((int)StreetStatus.Type.CarStreetWithLane);
        break;
    ...
}

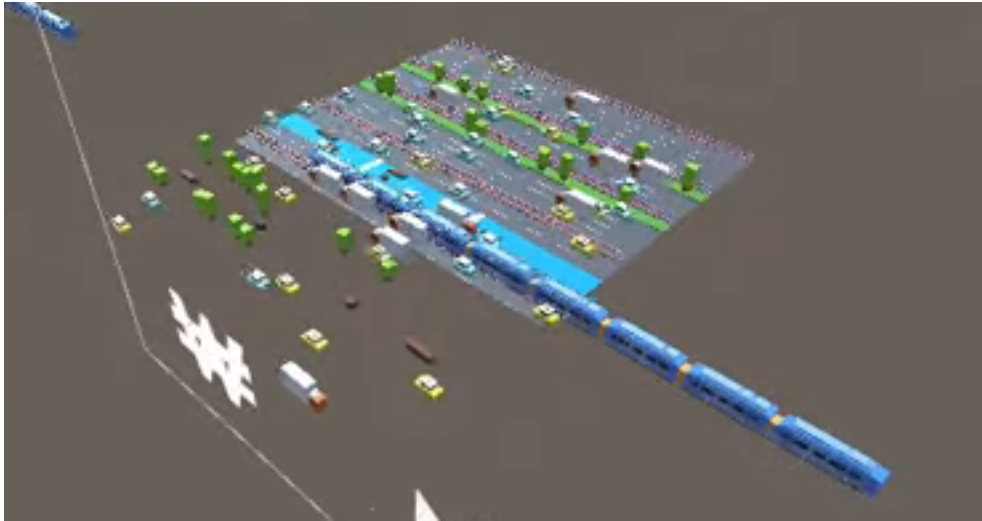
GameObject newStreet = Instantiate(GetRandomStreet(exceptions), genPosition, ...);
```

2.2 확률 보정으로 폭을 다양하게

규칙만 적용하면 지형이 한 줄씩만 나와 강과 도로의 폭이 늘 같아진다. 그래서 물이나 차도가 뿔뿔할 때 **정해진 확률로 같은 종류를 한 줄 더 붙이는** 보정을 넣었다. 덕분에 좁은 개울과 넓은 강, 2차선과 4차선 도로가 섞여 나와 구간마다 난이도가 달라진다.

```
// 물이면 일정한 확률로 하나를 더 만들(확률 보정)
if (newStreet.GetComponent<StreetStatus>().type == StreetStatus.Type.Water &&
    Random.Range(0f, 1f) <= waterGenrationRate)
{
    genPosition.z += streetInterval;
    GameObject revisedStreet = Instantiate(newStreet, genPosition, ...);
    farthestStreet = revisedStreet;
}
```

생성 기준점은 항상 `farthestStreet` — 지금까지 만든 것 중 가장 먼 지형이다. 새로 만든 지형이 다시 기준점이 되므로, 몇 줄을 붙이든 위치 계산이 어긋나지 않는다.

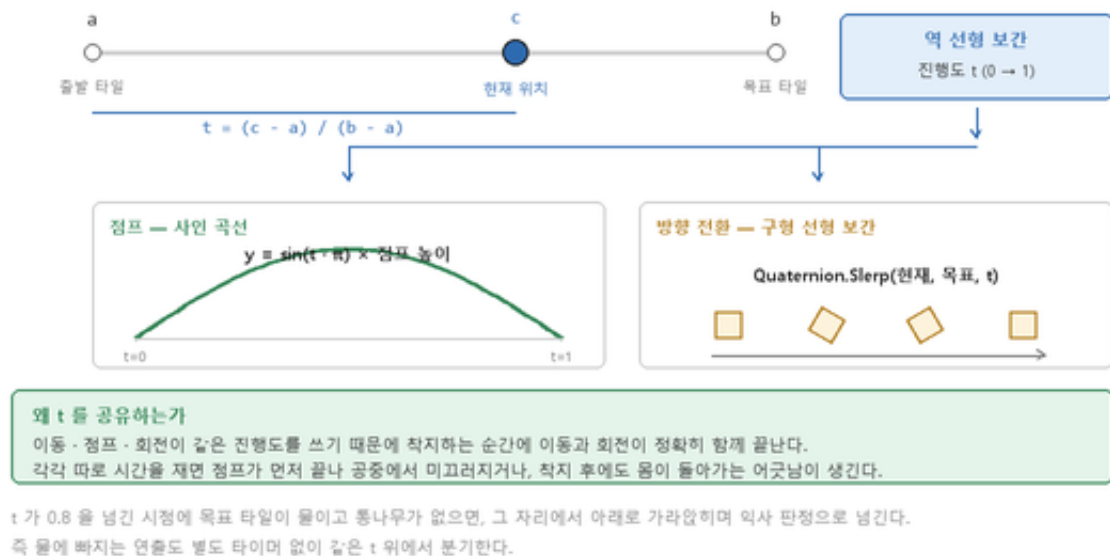


에디터에서 본 실제 생성 상태. 플레이어 주변으로 필요한 만큼만 존재하고 그 밖은 비어 있다.

3. 타일 단위 이동과 진행도 공유

캐릭터는 복셀 단위가 아니라 **타일 한 칸 단위로**, 그것도 타일 정중앙으로 이동한다. 그런데 원작의 이동을 보면 단순히 칸을 옮기는 것이 아니라 **점프하며 방향을 틀어** 넘어간다. 이동 · 점프 · 회전 세 가지가 동시에 진행되고, 착지하는 순간에 정확히 함께 끝나야 한다.

그림 2. 이동 진행도 t 하나로 점프와 회전을 함께 구동



3.1 역 선형 보간으로 진행도를 구한다

출발 타일 a 와 목표 타일 b, 그리고 현재 위치 c 를 알고 있으니, 선형 보간을 거꾸로 풀면 **지금 얼마나 갔는지**를 0~1 사이의 값 t 로 얻을 수 있다. 이 t 를 점프 높이와 회전에 그대로 넘기면 세 동작의 타이밍이 자동으로 일치한다. 각각 따로 시간을 재는 방식이라면 점프가 먼저 끝나 공중에서 미끄러지거나, 착지한 뒤에도 몸이 돌아가는 어긋남이 생긴다.

```
// y축 점프는 이동량에 비례해서 하므로 (c-a) / (b-a) 공식을 이용하여 t를 구한다.
a = tileToBeMoved.transform.position.z - (movement * direction.z);
b = tileToBeMoved.transform.position.z;
c = transform.position.z;
t = (c - a) / (b - a);

// 점프 — t를 180도로 정규화한 사인 곡선
transform.position = new Vector3(transform.position.x,
    tileToBeMoved.transform.position.y + Mathf.Sin(t * Mathf.PI) * jumpHeight,
    transform.position.z);

// 방향 전환 — 같은 t로 구형 선형 보간
transform.rotation = Quaternion.Slerp(transform.rotation, desiredRotation, t);
```

3.2 이동 전에 갈 곳을 먼저 확인한다

이동을 시작하기 전에 진행 방향으로 레이캐스트를 쏘아서 **그 칸의 타일 종류**를 확인한다. 나무처럼 막힌 칸이면 이동 자체를 취소하고, 물이면 이동은 하되 익사 판정 대상으로 넘긴다. 종류를 찾는 순서를 막힌 칸 → 물 → 일반으로 뒤서, 한 칸에 여러 콜라이더가 겹쳐 있어도 가장 강한 제약이 먼저 적용된다.

```
private void SetupMove(Vector3 _direction)
{
    TileStatus tile;

    if (CollisionTile(TileStatus.Kind.Block, _direction, movement, out tile))
        return; // 막혀 있으면 이동 취소

    if (CollisionTile(TileStatus.Kind.Water, _direction, movement, out tile))
        tileToBeMoved = tile;
    if (CollisionTile(TileStatus.Kind.General, _direction, movement, out tile))
        tileToBeMoved = tile;

    state = State.Move;
    direction = _direction;
    StreetGenerator.instance.Refresh(); // 한 걸음 = 지형 한 줄
    keyDelayTime = keyDelay; // 연타 억제
}
```

이동을 시작하는 이 지점에서 `StreetGenerator.Refresh()`를 호출한다. 지형 생성이 별도의 타이머나 거리 체크에 의존하지 않고 **플레이어의 걸음 수에 직접 묶여** 있어서, 앞쪽 여유분이 항상 일정하게 유지된다.

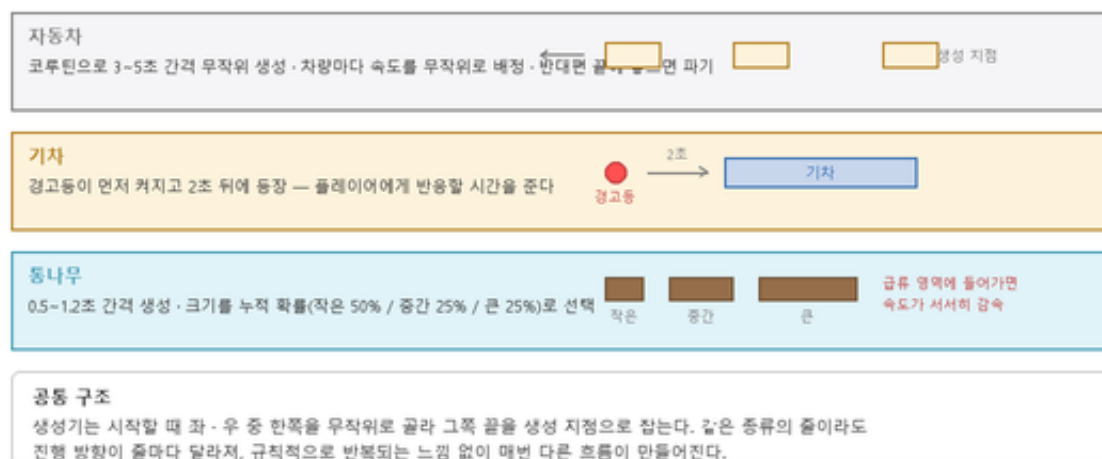
3.3 물 위에서의 분기

목표 타일이 물일 때는 같은 t 위에서 분기한다. t가 0.8을 넘기기 전까지는 평소처럼 점프 궤적을 따르고, 0.8을 넘긴 뒤에도 발 밑에 통나무가 없으면 아래로 가라앉기 시작해 일정 깊이에서 익사로 처리된다. 물에 **거의 닿는 순간**에 판정하므로, 통나무에 아슬아슬하게 올라타는 플레이어가 성립한다.

4. 장애물 3종

자동차 · 기차 · 통나무는 모두 '옆에서 흘러온다'는 점만 같고 생성 규칙은 서로 다르다. 같은 코드로 묶지 않고 각각의 성격에 맞게 따로 구현했다.

그림 3. 장애물 3종의 서로 다른 생성 방식



4.1 기차 — 예고 후 등장

기차는 자동차보다 훨씬 빠르기 때문에 그냥 등장시키면 피할 수 없다. 그래서 **경고등을 먼저 켜고 2초를 기다린 뒤** 생성한다. 플레이어가 반응할 시간을 주는 이 지연이 곧 난이도 조절 장치가 된다.

```
private IEnumerator GenerateTrain()
{
    signallampLight.SetActive(true); // 경고등 점등
    yield return new WaitForSeconds(ApproachedTime);
    signallampLight.SetActive(false);

    Train newTrain = Instantiate(trainPrefab.gameObject, generationPosition, ...);
    newTrain.direction = generationPositionSetter == 0 ? Vector3.right : Vector3.left;
}
```

4.2 통나무 — 크기별 확률과 급류

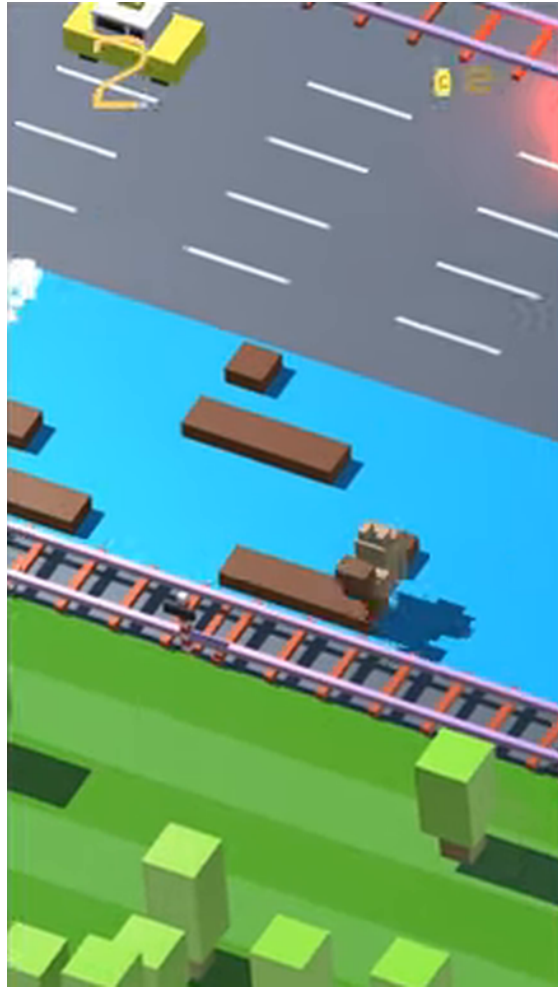
통나무는 크기가 세 종류이고, 누적 확률 구간으로 어느 크기가 나올지 결정한다 (작은 것 50%, 중간 25%, 큰 것 25%). 작은 통나무가 더 자주 나와야 긴장감이 유지되기 때문이다. 또 급류 영역에 들어간 통나무는 속도가 서서히 줄어들어, 같은 강에서도 흐름이 일정하지 않게 느껴진다.

```
float randomFactor = Random.Range(0, 1f);

if (randomFactor <= smallGenRate) newWood = woodPrefabs[0];
else if (randomFactor <= middleGenRate) newWood = woodPrefabs[1];
else if (randomFactor <= largeGenRate) newWood = woodPrefabs[2];
```

4.3 방향을 줄마다 다르게

세 생성기 모두 시작할 때 좌 · 우 중 한쪽을 무작위로 골라 그쪽 끝을 생성 지점으로 잡는다. 같은 종류의 줄이 여러 개 붙어도 **줄마다 흐르는 방향이 달라져**, 규칙적으로 반복되는 인상 없이 매번 다른 흐름이 만들어진다.



크기가 다른 통나무가 흘러오는 강 구간. 뒤쪽에는 예고등이 켜진 철길이 보인다.

5. 그 외 구현

5.1 나무 배치 — 셔플로 중복 제거

풀 구간의 나무는 무작위 좌표를 여러 번 뽑는 방식으로는 같은 칸이 중복으로 나와 실제 배치 수가 요청한 수보다 줄어든다. 그래서 **인덱스 배열**을 섞은 뒤 **앞에서부터 필요한 개수만큼 꺼내는** 방식을 썼다. 중복이 원천적으로 없고 요청한 개수가 그대로 배치된다.

그림 4. 셔플로 나무 위치를 겹치지 않게 배치

무작위로 좌표를 뽑으면



같은 칸이 중복으로 뽑혀 나무가 겹치고, 실제 배치 수가 요청한 수보다 줄어든다.

인덱스 배열을 섞어서 앞에서부터 꺼내면



0~29 인덱스 배열을 섞은 뒤 필요한 개수만큼 앞에서 꺼내므로 중복이 원천적으로 없고, 요청한 개수가 그대로 배치된다.

풀 구간은 나무 개수를 20~30개 사이에서 무작위로 정한 뒤 이 방식으로 뽑힌다.
특정 칸 하나는 항상 나무를 두어, 완전히 비어 있어 긴장감이 없는 구간이 나오지 않게 했다.

5.2 사망 연출

부딪힌 부위에 따라 연출을 나눴다. 차의 **측면**에 치이면 충돌 지점으로 옮겨진 뒤 차에 붙어 끌려가면서 납작하게 눌리고, **앞뒤**에 치이면 지면에 눌러 무작위 각도로 돌아간다. 물에서 타이밍을 놓치면 가라앉으면서 카메라가 서서히 줌인된다.

5.3 점수와 재시작

점수는 전진한 거리(z 좌표)를 타일 크기로 나눈 값이고, 뒤로 물러나도 줄지 않도록 최댓값만 갱신한다. 최고 기록은 `PlayerPrefs` 에 저장하고, 사망 시 3초 뒤 씬을 다시 불러오는 방식으로 재시작한다.

5.4 카메라

카메라는 `LateUpdate` 에서 목표 위치로 선형 보간해 따라간다. 좌우 이동은 **일정 범위를 넘지 않도록 제한**을 걸어, 플레이어가 옆으로 많이 움직여도 화면이 지형 밖을 비추지 않게 했다.

6. 회고

이 작품에서 가장 크게 배운 것은 **하나의 값으로 여러 동작을 묶는 것**의 효과였다. 이동 진행도 `t` 를 점프와 회전이 함께 쓰게 하자, 타이밍을 맞추는 코드가 아예 필요하지 않았다. '동시에 끝나야 하는 것들은 같은 기준을 공유하게 한다' 는 원칙을 이때 몸으로 익혔다.

지형 생성에서는 **무작위성에 규칙을 씌우는** 작업이 핵심이었다. 완전한 무작위는 플레이할 수 없는 맵을 만들어내기 때문에, 제외 목록으로 하한을 보장하고 확률 보정으로 다양성을 얻는 두 단계 구조가 필요했다.

지금 다시 만든다면 고칠 부분도 있다. 타일 크기와 지형 폭 같은 값이 여러 스크립트에 **숫자로 흩어져** 있어 한 곳을 바꾸면 다른 곳이 어긋났다. 상수를 한 곳에 모으거나 설정 오브젝트로 뺏어야 했다. 또 자동차 · 통나무를 매번 생성하고 파기하는 대신 오브젝트 풀을 썼다면 모바일에서의 부담을 줄일 수 있었을 것이다.

스크립트 구성

`StreetGenerator` 지형 생성과 제외 규칙 · 확률 보정 · `StreetStatus` 지형 종류 · `TileAllocator` 타일 배치

`PlayerStatus` 상태 · 이동 · 진행도 · 익사 판정 · `PlayerCrash` 부위별 사망 연출

`CarGenerator` / `TrainGenerator` / `WoodGenerator` / `GrassGenerator` 각 장애물 · 장식물 생성

`Car` / `Train` / `Wood` / `Tree` 개별 이동과 파기 · 급류 감속 · 흔들림

`GameManager` 점수 · 최고 기록 · 재시작 · `CameraFollow` 추적과 범위 제한